

Name - Harshit Vats
Superset Id - 7748373
VIT Bhopal

Spring-Core-Maven

Exercise 1:Configuring a Basic Spring Application

Objective

Configure a basic Spring application for a library management system using Spring Framework and Maven.

Steps

1. A Maven project named LibraryManagement is created using start.spring.io.
2. Spring Core dependencies are added in the pom.xml file including spring-context, spring-aop, and spring-webmvc.
3. An XML configuration file named applicationContext.xml is created in src/main/resources directory.
4. Beans for BookService and BookRepository are defined in the XML file using <bean> tags.
5. A package com.library.service is created with a class BookService containing a setter method for BookRepository.
6. A package com.library.repository is created with a class BookRepository containing a method findBookById().
7. A main class LibraryManagementApplication loads the Spring context using ClassPathXmlApplicationContext and retrieves the BookService bean.
8. The application is executed and the output is verified in the console.

Output:

```
genc-dn5.0/week2/SpringCore-Maven on J main [!?] is v1.0-SNAPSHOT via v26.0.1 took 22s
• > mvn compile exec:java -Dexec.mainClass="com.library.LibraryManagementApplication"
[INFO] Scanning for projects...
[INFO] -----< com.library:LibraryManagement >-----
[INFO] Building LibraryManagement 1.0-SNAPSHOT
[INFO] from pom.xml
[INFO] -----[ jar ]-----
[INFO] --- resources:3.4.0:resources (default-resources) @ LibraryManagement ---
[WARNING] Using platform encoding (UTF-8 actually) to copy filtered resources, i.e. build is platform dependent!
[INFO] Copying 1 resource from src/main/resources to target/classes
[INFO] --- compiler:3.11.0:compile (default-compile) @ LibraryManagement ---
[INFO] Nothing to compile - all classes are up to date
[INFO] --- exec:3.6.3:java (default-cli) @ LibraryManagement ---
Book Details: Spring in Action - Craig Walls
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 1.033 s
[INFO] Finished at: 2026-07-02T21:35:22+05:30
[INFO] -----
```

Exercise 2: Implementing Dependency Injection

Objective

Implement dependency injection between BookService and BookRepository classes using Spring's IoC and DI.

Steps

1. The applicationContext.xml file is modified to wire BookRepository into BookService using the <property> tag with ref attribute.
2. The BookService class contains a setter method setBookRepository() for the BookRepository dependency.
3. The LibraryManagementApplication main class is executed to verify that dependency injection works correctly.
4. The Spring container automatically injects the BookRepository instance into BookService without manual instantiation.

Key Concepts

- IoC (Inversion of Control): The Spring container takes control of creating objects and managing their dependencies.
- Dependency Injection (DI): Spring automatically injects required dependencies into beans through setter methods or constructor.
- Setter Injection: When Spring sees <property> tag in XML, it calls the corresponding setter method.
- The <property name="bookRepository" ref="bookRepository"/> line tells Spring to inject the bookRepository bean into the bookService bean.

Output:

```
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/resolver/maven-resolver-api/1.9.24/maven-resolver-api-1.9.24.jar (157 kB at 920 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/ow2/asm/asm/9.9.1/asm-9.9.1.jar (126 kB at 454 kB/s)
Book Details: Spring in Action - Craig Walls
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 20.554 s
[INFO] Finished at: 2026-07-02T21:29:51+05:30
[INFO] -----
```

Exercise 4: Creating and Configuring a Maven Project

Objective

Create and configure a Maven project for the library management application with required Spring dependencies.

Steps

1. A new Maven project named LibraryManagement is created with groupId as com.library and artifactId as LibraryManagement.
2. The following Spring dependencies are added in the pom.xml file:
 - spring-context - For Spring IoC container
 - spring-aop - For Aspect-Oriented Programming
 - spring-webmvc - For Spring MVC support
3. The Maven Compiler Plugin is configured for Java version 1.8 in the pom.xml file under <build><plugins> section.
4. The project structure is verified to contain src/main/java, src/main/resources, and src/test/java directories.

Key Concepts

- pom.xml is the heart of any Maven project
- <properties> section defines project-wide configuration values
- <dependencies> section declares all required libraries
- <plugins> section configures build tools
- Maven Compiler Plugin controls which Java version is used for compilation
- Property variables like \${spring.version} help manage dependency versions

Output:

```
genc-dn5.0 on 17 main [!?]
> cd week2/SpringCore-Maven && mvn compile exec:java -Dexec.mainClass="com.library.LibraryManagementApplication" -q
Book Details: Spring in Action - Craig Walls
```

